Kubernetes

What is Kubernetes (K8s)?

Kubernetes is a system that manages your applications automatically.

More precisely:

Kubernetes is a **container orchestration platform** that **deploys, scales, heals, and manages containerized applications**.

- You give it your app
- Kubernetes runs it **reliably on multiple machines**
- Keeps it alive even if things crash

Before Kubernetes, you had:

1. Just code

You wrote backend in Node.js → ran with:

```
node server.js
```

2. Then came Docker (containers)

You wrapped your app into a container:

```
docker run my-app
```

Now it's portable.

❗ **Problem starts here...**

Running **1 container = easy**

Running **100 containers = chaos**

Ask yourself:

- What if server crashes?
- How to scale when traffic spikes?
- How do containers talk to each other?

- How to update app without downtime?
- How to restart failed containers?

👉 This is where Kubernetes comes in.

Why Kubernetes is Needed

🧩 Problem 1: Manual Deployment is Pain

Without Kubernetes:

- SSH secure shell into server
- Run Docker manually
- Pray nothing crashes

With Kubernetes:

- You say:

“Run 3 copies of my app”

Kubernetes does everything.

⚙️ Problem 2: Scaling

Traffic spike (say IPL match 😄):

Without K8s:

- Manually start more containers

With K8s:

- Auto-scale:

CPU ↑ → add more containers

💀 Problem 3: Crashes

Without K8s:

- App crashes → dead

With K8s:

- Auto-healing:

Container dies → Kubernetes restarts it

🌐 Problem 4: Networking

Without K8s:

- Hardcoded IPs
- Messy service communication

With K8s:

- Built-in service discovery:

“backend-service” → auto routing

🔄 Problem 5: Zero Downtime Deployments

Without K8s:

- Stop old → start new → downtime

With K8s:

- Rolling updates:

Replace containers gradually

🧠 Mental Model (THIS IS GOLD)

Think of Kubernetes like:

👉 Operating System for your servers

OS (like Windows/Linux)	Kubernetes
Runs programs	Runs containers
Manages CPU/RAM	Manages cluster resources

OS (like Windows/Linux)	Kubernetes
Restarts crashed apps	Restarts failed containers
Handles processes	Handles pods

📐 Real World Example (Your Backend Mindset)

Let’s say you built:

- Booking Service (Node.js)
- Event Service
- Redis
- MongoDB

Now imagine:

10,000 users booking tickets

Without K8s → nightmare

With K8s:

- 5 instances of booking service
- 3 instances of event service
- Redis managed
- Auto scaling
- Crash recovery

👉 This is production backend engineering.

⚡ Core Concept (Important Term)

👉 Pod

Smallest unit in Kubernetes

- Runs your container
- Usually 1 app per pod

⚠️ Reality Check (Important)

Let me challenge your thinking (as you asked me to):

You might think:

"Why not just use Docker + PM2 + Nginx?"

That works for:

- Small apps
- Single server

But fails when:

- Multi-server deployment
- Auto scaling needed
- Microservices grow
- High availability needed

👉 Kubernetes solves *distributed systems problems*

☸ BEFORE KUBERNETES (Real World Setup)

Let's say you built:

- Booking Service (Node.js)
- Event Service
- Redis
- MongoDB

🏗 Deployment Flow (Old Way)

Step 1: Get a server

You rent 1–2 VMs (AWS EC2)

Step 2: Setup everything manually

ssh into server
install node
install nginx
install pm2
install docker (maybe)

🔥 Running Your App

pm2 start server.js

or

docker run my-app

💣 Problems Start Here

❗ 1. Scaling is Manual & Slow

Traffic spike:

You:

- Launch new EC2
- SSH
- Setup everything again

⛔ Takes minutes → users already gone

❗ 2. No Auto-Healing

If app crashes:

server down 😬

Unless:

- PM2 restarts (basic)
- But if **server dies → everything gone**

❗ 3. Load Balancing is DIY

You configure:

- Nginx
- Or AWS ELB

But:

- You manually add/remove servers
- No dynamic awareness

❗ 4. Deployment = Risky

Deploy new version:

stop old app
start new app

⛔ Downtime
⛔ Bugs = rollback pain

❗ 5. Microservices = Chaos

Now imagine:

- booking-service (3 instances)
- event-service (2 instances)

Problems:

- Hardcoded IPs
- Service discovery messy
- Networking pain

❗ 6. Resource Wastage

Server has:

- 8GB RAM

App uses:

- 1GB

⛔ Remaining wasted

🧠 Summary (Before K8s)

You are: A human orchestrator 😵 Handling:

- Deployment
- Scaling
- Failures
- Networking
- Resource management

🚀 AFTER KUBERNETES

Now same system with Kubernetes.

📐 New Approach

Instead of commands, you write **desired state**:

I want 3 instances of booking-service
I want auto-scaling
I want zero downtime

Kubernetes handles everything.

🔥 What Changes Fundamentally

⚙️ 1. You Stop Managing Servers

Before:

“Run app on this server”

After:

“Run app anywhere in cluster”

👉 Kubernetes decides *where*

📈 2. Scaling Becomes Automatic

You define:

replicas: 3

Traffic increases:

👉 Kubernetes creates more pods

💀 3. Self-Healing System

Pod crashes?

👉 Kubernetes:

- Detects failure
- Restarts it
- Maintains desired count

No human needed.

🌐 4. Built-in Networking

You say:

service: booking-service

Now:

- Other services call it via name
- No IP management

🔁 5. Zero Downtime Deployments

Update app:

👉 Kubernetes:

- Gradually replaces pods
- Keeps system running

📦 6. Efficient Resource Usage

Kubernetes packs containers smartly:

- Uses CPU/RAM efficiently
- Multiple apps per node

🧠 Mental Shift (VERY IMPORTANT)

BEFORE:

“I run containers”

AFTER:

“I describe system → Kubernetes runs it”

🧠 Kubernetes Architecture (Big Picture)

What is a Cluster (in Kubernetes)?

A Kubernetes cluster is:

A group of machines working together as one system to run and manage your applications.

A cluster has 2 main parts:

1. Control Plane (Brain)

2. Worker Nodes (Muscle)

Kubernetes = **Control Plane (brain)** + **Worker Nodes (muscles)**

📐 Visual Mental Model

```
[ Control Plane ]
 ├── API Server
 ├── Scheduler
 ├── Controller Manager
 └── etcd (DB)

        ↓

[ Worker Nodes ]
 ├── Pod (your app)
 ├── kubelet
 └── container runtime
```

🧠 1. CONTROL PLANE (Master Node)

This is the **brain of Kubernetes**.

👉 It decides:

- What should run
- Where it should run
- When to restart things

🔥 1. API Server (ENTRY POINT)

This is the **front door of Kubernetes**

Every action goes through this. You run:

kubectl apply -f app.yaml

👉 Request goes to **API Server**

Responsibilities:

- Accept requests (kubectl / UI / CI/CD)
- Validate configs
- Store state in **etcd**
- Trigger other components

🧠 Think Like Backend

👉 API Server = **Express server of Kubernetes**

- Receives request
- Validates
- Calls services

⚙️ 2. etcd (DATABASE)

The **source of truth** of Kubernetes

Stores everything:

- Pods

- Deployments
- Services
- Cluster state

You say:

replicas: 3

👉 Stored in etcd

Important Insight:

Kubernetes works on **Desired State vs Actual State**

- Desired → stored in etcd
- Actual → what's running

👉 System constantly tries to match both

🎯 3. Scheduler

Decides **WHERE your pod runs**

🧪 Example

You create a pod.

Scheduler checks:

- Which node has free CPU?
- Which node has memory?
- Any constraints?

👉 Then assigns pod to a node

🧠 Analogy

👉 Scheduler = **Load balancer for machines**

🔄 4. Controller Manager

Ensures system matches desired state

🧪 Example

You want:

replicas: 3

But only 2 running.

👉 Controller:

- Detects mismatch
- Creates 1 more pod

Types of Controllers:

- Replica Controller
- Node Controller
- Job Controller

🧠 Analogy

👉 Controller = **Auto-correct system**

💪 2. WORKER NODES

These actually **run your application**

📦 What's inside a Worker Node?

---

1. Pod

Smallest unit in Kubernetes

- Contains your container(s)
- Runs your app

2. kubelet

Agent running on each node

Responsibilities:

- Talks to API Server
- Ensures pods are running
- Reports status

🧠 **Analogy**

👉 kubelet = **local manager on each machine**

3. Container Runtime

Examples:

- Docker (older)
- containerd (modern)

👉 Actually runs containers

🍼 **Step-by-Step Flow**

Step 1:

You run:

kubectl apply -f app.yaml

Step 2:

👉 API Server receives request

- Validates YAML
- Stores in **etcd**

Step 3:

👉 Scheduler sees new pod

- Finds best node
- Assigns pod

Step 4:

👉 kubelet (on that node):

- Gets instruction
- Pulls Docker image
- Starts container

Step 5:

👉 Controller keeps checking:

- If pod dies → restart
- If less replicas → create more

🧠 ONE LINE SUMMARY

You declare → Kubernetes figures out → Nodes execute

⚔️ Challenge Your Thinking

“Why so many components? Why not one system?”

Because:

👉 Kubernetes solves **distributed systems problems**

Separation gives:

- Scalability
- Fault tolerance
- Modularity

🚨 **Real Production Insight**

- Control Plane can run on multiple machines (HA)
- etcd is critical (if lost → cluster gone)
- Scheduler decisions affect performance heavily

⚡ **What Most People Miss**

This is key:

Kubernetes is a **reconciliation system**

Loop:

Check → Compare → Fix → Repeat

Constantly.

Resource management

👉 You **don’t assign resources manually to machines**
👉 You **declare requirements**, and Kubernetes decides everything

1. You define requirements (in Pod YAML)

Example:

```
resources:
  requests:
    cpu: "500m"
    memory: "256Mi"
  limits:
    cpu: "1"
    memory: "512Mi"
```

Meaning:

- **request** = minimum needed (guarantee)
- **limit** = max allowed

2. API Server stores it

You send this config to the cluster via:

- kubectl
- or API call

👉 API Server saves it in **etcd**

3. Scheduler decides WHERE to run

Scheduler checks:

- Which node has enough CPU + memory?
- Which node is least loaded?

👉 Then assigns your Pod to a node

4. Kubelet (on node) enforces it

On that node:

- Kubelet receives instruction

- Starts container using runtime (like Docker)

5. Runtime enforces limits

This is crucial:

- CPU → throttled if exceeded
- Memory → container **killed (OOMKilled)** if exceeded

👉 This is enforced by Linux (cgroups)

Overcommit in simple terms

👉 **You promise less CPU than you actually have, but allow apps to use more if available.**

In a **Kubernetes cluster**:

- Node has **4 CPU**
- You schedule pods with:
    - total **requests = 2 CPU**
    - total **limits = 6 CPU**

👉 This is overcommit

Overcommit = **"Assume not everyone will use full power at the same time."**

🔥 **Why companies use this**

Without overcommit:

- 4 CPU machine → only 4 CPU worth apps
  👉 Waste of resources

With overcommit:

- Same machine → runs more apps
  👉 Better utilization 💰

⚡ **What actually happens**

👉 **You will NOT get 5–6 CPU**

Because:

The machine physically has only **4 CPU**

🔥 **So what does Kubernetes do?**

Using Linux (cgroups), it enforces:

👉 **CPU Throttling**

- All pods **compete** for CPU
- No one gets full demand
- Everyone gets a **slice**

🧠 **1. QoS Classes**

1. 🟢 **Guaranteed (VIP)**
   - **You said exactly:**
     - **“I need 1 CPU, 1GB RAM”**
     - **And limit is also same**

👉 **Kubernetes trusts you**

✅ **Most stable**
❌ **Killed last**

2. 🟡 **Burstable (Regular)**
   - **You said:**
     - **“I need at least 1GB, but I *might* use more”**

👉 **Flexible user**

⚖️ **Medium priority**
⚠️ **Killed if pressure increases**

3. 🔴 BestEffort (Free users)

- **You said nothing (no requests/limits)**

👉 Kubernetes thinks:

"This guy is just freeloading 😅"

❌ First to be killed

⚠️ When memory pressure happens

Imagine RAM is full:

👉 Kubernetes starts removing people:

1. **BestEffort ❌**
2. **Burstable ❌**
3. **Guaranteed (last option)**

2. Resource Fragmentation (this is VERY real)

This one hurts in production.

🧠 Problem in simple terms

You have enough total resources… but not in the right shape

⚡ Kubernetes version

Cluster:

- **Node A → 2 CPU free**
- **Node B → 2 CPU free**

Pod needs: ➔ 3 CPU

👉 **What happens?**

❌ **It won’t split across nodes**
❌ **It won’t combine resources**

👉 **Pod stays Pending**

⚠️ **Key truth**

Kubernetes schedules per node, not across nodes

Series of events

1️⃣ **You write a manifest**

2️⃣ **Apply manifest**

kubectl apply -f deployment.yaml

What happens:

1. **kubectl sends request to API server**
2. **API server validates YAML and stores desired state in etcd**
3. **Controllers (Deployment Controller) see desired state**

3️⃣ **Deployment creates ReplicaSet**

- **Deployment automatically creates a ReplicaSet**
- **ReplicaSet knows how many pods should exist**

4️⃣ **ReplicaSet creates Pods**

- **Scheduler assigns Pods to nodes**
- **Kubelet on each node runs the container(s)**

5️⃣ **Pods start containers**

- **Containers start listening on containerPort**
- **Pod gets its internal IP**
- **Health checks (liveness/readiness probes) run**

6️⃣ **Create Service (to expose pods)**

- **Service selects pods via labels**
- **Assigns stable IP / DNS name**
- **External access via NodePort**

7️⃣ **Access app**

curl http://<minikube-ip>:31000

- **Traffic goes through Service → Pod → Container**

- Load balancing handled automatically across replicas

8️⃣ Updates (Rolling Update)

- Change Deployment image to v2
- kubectl apply -f deployment.yaml
- Deployment creates new ReplicaSet
- Old pods replaced gradually → zero downtime

9️⃣ Scaling

kubectl scale deployment booking-deployment --replicas=5

- Deployment updates ReplicaSet → new pods created
- Extra pods start receiving traffic automatically

1️⃣0️⃣ Deletion

- kubectl delete deployment booking-deployment
- Deletes Deployment → ReplicaSet → Pods
- kubectl delete service booking-service
- Service is removed, external access gone

🧠 Mental Model: Flow of Events

Manifest YAML
↓
kubectl apply
↓
API Server → stores desired state in etcd
↓
Deployment Controller
↓

ReplicaSet
↓
Pods (Scheduler → Kubelet → Containers)
↓
Service exposes Pods
↓
User/browser → Service → Pod → Container

🔥 **Key Insights**

1. **kubectl never creates pods directly**
2. **Deployment → ReplicaSet → Pods**
3. **Service = stable access point**
4. **All actions are declarative (you tell Kubernetes desired state)**

📦 What is a Pod (REAL understanding)

A Pod is the smallest deployable unit in Kubernetes that wraps one or more containers with shared context.

That "shared context" is the key.

"Why not just run containers directly?"

Because Kubernetes is not managing *containers*, it's managing application units.

👉 A Pod = "how this app should run"

🔥 Why Pods (NOT Containers)

❗ Problem with raw containers

Containers are:

- **Isolated**
- **Stateless (mostly)**
- **Independent**

But real apps often need:

- **Shared storage**
- **Tight communication**
- **Same lifecycle**

✅ Pod solves this

Inside a Pod:

- **Containers share network (same IP)**
- **Containers share storage (volumes)**
- **Start/stop together**

🧠 **Golden Line**

Pod = wrapper around containers + shared environment

🧪 **Example (IMPORTANT)**

Imagine:

- **Your main app (Node.js)**
- **A logging agent**

Without Pod:

- **Hard to sync them**
- **Separate networking**

With Pod:

```
containers:
  - name: app
    image: node-app
  - name: logger
    image: log-agent
```

👉 **Both:**

- **Run together**
- **Talk via localhost**
- **Share data**

⚠️ **Reality Check**

In 90% cases:

👉 **1 Pod = 1 Container**

Multi-container pods are for special patterns, not default.

🔄 Pod Lifecycle (VERY IMPORTANT)

Pods are NOT permanent

They are:

Disposable, replaceable units

📊 Lifecycle Phases

---

1. Pending

- **Pod created**
- **Not yet scheduled or pulling image**

---

2. Running

- **Container is running**
- **App is live**

---

3. Succeeded

- **Completed successfully (jobs)**

---

4. Failed

- **Crashed / error**

---

5. Unknown

- **Node issue**

🧠 Key Insight

Pods are NOT restarted—they are REPLACED

This is huge.

❗ **Example**

Pod crashes:

👉 **Kubernetes:**

- **Kills pod**
- **Creates NEW pod**

Not “restart same process”

You say:

replicas: 3

What happens:

1. **3 Pods created**
2. **One crashes ❌**
3. **Controller detects mismatch**
4. **New Pod created ✅**

👉 **This is self-healing**

⚙️ **Multi-Container Pods (ADVANCED but IMPORTANT)**

Used when containers are tightly coupled

🧩 **Pattern 1: Sidecar (MOST IMPORTANT)**

Main app + helper container

Example:

- App → Node.js server
- Sidecar → logging / monitoring

```
containers:
  - name: app
    image: booking-service
  - name: sidecar
    image: log-collector
```

👉 Sidecar enhances app

🧩 Pattern 2: Init Container

Runs before main container starts

Example:

- Wait for DB
- Run migrations

```
initContainers:
  - name: init-db
    image: busybox
```

👉 Ensures system is ready

🧩 Pattern 3: Ambassador

Acts as proxy

Example:

- Handle external API communication

🧠 Shared Resources in Pod

🌐 Network

All containers share:

localhost

👉 No need for service discovery inside pod

💾 Storage

Shared volumes:

```
volumes:
  - name: shared-data
```

👉 Containers read/write same data

If containers don’t need to share lifecycle → DON’T put them in same pod

❌ Bad Example

- **Booking service**
- **Payment service**

👉 Separate Pods

🧠 Mental Model (Strong One)

Think:

Pod = Process group in OS

- **Containers = threads/processes**
- **Shared resources = memory/network**

⚡ What Beginners Get Wrong

❌ Mistake 1:

Thinking Pod = container

👉 Wrong abstraction level

❌ Mistake 2:

Trying to “restart pod”

👉 Kubernetes replaces, not restarts

❌ Mistake 3:

Overusing multi-container pods

👉 Keep it simple unless needed

Creating a Pod

Because:

Pods are temporary objects
Controllers (Deployment) manage them

pod.yaml

Create Pod (manual)

```
apiVersion: v1
kind: Pod
metadata:
  name: booking-pod
spec:
  containers:
    - name: booking-app
      image: booking-app
```

```
    ports:
      - containerPort: 3000
```

Create a pod

```
kubectl apply -f pod.yaml
```

Delete Pod

```
kubectl delete pod booking-pod
```

❌ **Problem with this approach**

- **No auto-healing**
- **No scaling**
- **If pod dies → gone**

Manifest

🧠 **What is a Kubernetes Manifest?**

👉 **A manifest is just a YAML file that defines what you want Kubernetes to create**

Kubernetes manifest = declarative configuration file for resources

⚙️ **Example (you already used)**

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: booking-deployment
spec:
  replicas: 2
  ...
```

👉 **This file = manifest**

🧠 **What "declarative" means**

You say:

replicas: 2

👉 **You're NOT saying:**

- **how to create pods** ❌
- **how to schedule** ❌

👉 **You're saying:**

"I want 2 pods"

Kubernetes figures out how

🔧 What can be a manifest?

Any Kubernetes resource:

- **Pod**
- **Deployment**
- **Service**
- **ConfigMap**
- **Secret**

📦 Common Structure (ALWAYS SAME)

```
apiVersion: <which API>
kind: <what resource>
metadata:
  name: <name>
spec:
  <desired state>
```

🧠 Meaning of each

---

apiVersion

👉 **Which version of API to use**

kind

👉 **Type of resource**

Examples:

- **Pod**
- **Deployment**
- **Service**

metadata

👉 **Info about object (name, labels)**

spec

👉 **Desired state (MOST IMPORTANT)**

⚔️ **What happens when you run**

kubectl apply -f deployment.yaml

Manifest → kubectl → API Server → etcd
→ Controllers → actual resources

🔥 **Key Insight**

Manifest = “what you want”
Kubernetes = “makes it happen”

”

🧠 **Imperative vs Declarative**

❌ **Imperative (not preferred)**

kubectl run app --image=nginx

✅ **Declarative (manifest)**

kubectl apply -f deployment.yaml

👉 **Declarative = scalable + version-controlled**

⚔️ **Why manifests are important**

- **Version control (Git)**
- **Reproducible deployments**
- **Same file → local + production**

Replicaset

🧠 **What is a ReplicaSet?**

👉 **A ReplicaSet ensures a fixed number of Pods are always running**

🔥 **One-line definition**

ReplicaSet = keeps N identical pods alive

⚙️ **Example**

replicas: 3

👉 **ReplicaSet ensures:**

Always 3 pods running

🧠 **What it actually does**

If a pod dies ❌

Pod deleted → ReplicaSet creates new pod

If you delete manually ❌

kubectl delete pod xyz

👉 **ReplicaSet:**

“Need 3 → only 2 → create 1 more”

If you scale up 📈

kubectl scale deployment booking-deployment --replicas=5

👉 **ReplicaSet:**

Creates 2 more pods

⚔️ **Important Truth**

👉 **You don’t usually create ReplicaSet directly**

Because:

Deployment → manages ReplicaSet → manages Pods

🔥 **Hierarchy**

Deployment
↓
ReplicaSet
↓
Pods

⚙️ **Real Flow**

When you run:

kubectl apply -f deployment.yaml

👉 **Kubernetes does:**

1. **Create Deployment**
2. **Deployment creates ReplicaSet**
3. **ReplicaSet creates Pods**

🔍 **How to see it**

kubectl get rs

👉 **You’ll see something like:**

booking-deployment-7c9f8d

🧠 **Selector Logic (IMPORTANT)**

ReplicaSet uses labels:

```
matchLabels:
  app: booking
```

👉 **It only manages pods with this label**

⚠️ **Critical Rule**

If labels mismatch:

👉 **ReplicaSet won't control pods**

🔥 **Real Power**

ReplicaSet ensures:

- **High availability**
- **Fault tolerance**
- **Consistency**

⚔️ **Why Deployment uses ReplicaSet?**

For:

👉 **Rolling updates**

Example:

You update image:

```
image: booking-app:v2
```

👉 **Deployment:**

Creates new ReplicaSet (v2)
Gradually kills old pods (v1)

🧠 Final Mental Model

ReplicaSet = “maintain count”
Deployment = “manage versions + updates”

🤔 How to create a ReplicaSet

1️⃣ Direct creation (not recommended in real world)

You can create a ReplicaSet YAML manually:

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: booking-rs
spec:
  replicas: 3
  selector:
    matchLabels:
      app: booking
  template:
    metadata:
      labels:
        app: booking
    spec:
      containers:
        - name: booking-app
          image: booking-app:v1
          ports:
            - containerPort: 3000
```

```
kubectl apply -f replicaset.yaml
```

☑️ What happens

- **Kubernetes creates ReplicaSet object**

- **ReplicaSet ensures 3 pods are running**
- **If pods die → ReplicaSet recreates them**

2️⃣ But in real-world practice

You almost never create ReplicaSet manually.

DEPLOYMENTS (How Pods are managed)

🧠 **First Truth (Important)**

You almost NEVER create Pods directly in real apps.

Instead:

👉 **You create a Deployment**
👉 **Deployment manages Pods for you**

What is a Deployment?

A Deployment is a controller that ensures a certain number of Pods are always running

🧪 **Simple Example**

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: booking-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: booking
  template:
    metadata:
      labels:
        app: booking
    spec:
      containers:
        - name: app
          image: booking-service
```

```
    ports:
      - containerPort: 3000
```

🔥 **What this means**

You are saying:

“I want 3 instances of my booking app always running”

⚙️ **What Kubernetes does**

- **Creates 3 Pods**
- **Monitors them**
- **Replaces if any die**
- **Keeps count = 3 always**

🧠 **Mental Model**

👉 **Deployment = manager**
👉 **Pods = workers**

🔄 **Key Features of Deployments**

✅ **1. Self-Healing**

Pod dies? 👉 Deployment creates a new one

✅ **2. Scaling**

Change:

replicas: 5 👉 Kubernetes creates 2 more Pods

✅ **3. Rolling Updates (🔥 VERY IMPORTANT)**

You update your app: image: booking-service:v2

What Kubernetes does:

Instead of:

❌ Kill all → start new (downtime)

It does:

Old Pod → New Pod → Old removed
Old Pod → New Pod → Old removed

👉 One by one replacement

🧠 Result:

- **No downtime**
- **Safe deployment**

✅ 4. Rollback

Bug in v2?

kubectl rollout undo deployment booking-deployment

👉 Back to v1 instantly

🌐 PART 2: SERVICES (Networking Between Pods)

🧠 Problem First

Pods are:

- **Dynamic**
- **Ephemeral (die & recreate)**
- **IP keeps changing**

❗ Without Service

You have 3 Pods:

Pod1 → 10.0.0.1
Pod2 → 10.0.0.2
Pod3 → 10.0.0.3

If one dies:

Pod2 gone → new Pod → 10.0.0.9

👉 **Your system breaks if using IPs**

🎯 **What is a Service?**

A Service is a stable network endpoint to access Pods

🧪 **Example**

```
apiVersion: v1
kind: Service
metadata:
  name: booking-service
spec:
  selector:
    app: booking
  ports:
    - port: 80
      targetPort: 3000
  type: ClusterIP
```

🔥 **What this does**

- **Finds Pods with label app: booking**
- **Routes traffic to them**
- **Provides stable name**

🧠 **Now you can call:**

fetch("http://booking-service")

👉 **Kubernetes handles routing**

🔁 **Built-in Load Balancing**

Service automatically:

Request → Pod1
Request → Pod2
Request → Pod3

👉 **No Nginx needed internally**

🔥 **Types of Services (Important)**

---

1. ClusterIP (default)

- **Internal communication**
- **Used between services**

---

2. NodePort

- **Exposes service on node port**
- **For testing**

---

3. LoadBalancer

- **Exposes to internet**
- **Uses cloud provider**

🧠 **Mental Model**

Without Kubernetes:

Frontend → Nginx → Backend IPs

With Kubernetes:

Frontend → Service → Pods

👉 **Service = smart router**

🔥 **REAL FLOW (Put It Together)**

🍼 **You deploy:**

- **Deployment (3 pods)**
- **Service (expose them)**

🧠 **Flow:**

Client → Service → Pod → Response

If Pod dies:

👉 **Deployment creates new one**
👉 **Service automatically routes to new pod**

👉 **System never breaks**

⚔️ **Challenge Your Thinking**

You might think:

“Service is just load balancer?”

Not fully.

👉 **It also:**

- **Provides stable DNS**
- **Handles service discovery**
- **Decouples pods from clients**

Deployment

- **Manages pods**
- **Ensures desired number of replicas**
- **Handles updates / rolling updates / self-healing**

🔹 Step 1: Create Deployment YAML

Example for your booking app:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: booking-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: booking
  template:
    metadata:
      labels:
        app: booking
    spec:
      containers:
        - name: booking-app
          image: booking-app:v1
          ports:
            - containerPort: 3000
```

✅ **Key Points**

1. **replicas: 2 → number of pods**
2. **selector.matchLabels must match template.metadata.labels**
3. **containers.image → your Docker image**
4. **containerPort → internal port of container**

🔹 **Step 2: Apply Deployment**

```
kubectl apply -f deployment.yaml
```

- **Creates a Deployment**
- **Creates a ReplicaSet automatically**
- **ReplicaSet creates pods**

🔹 **Step 3: Verify**

```
kubectl get deployments
kubectl get pods
kubectl describe deployment booking-deployment
```

🔹 **Step 4: Scaling (Optional)**

```
kubectl scale deployment booking-deployment --replicas=5
```

- **Deployment updates ReplicaSet**
- **ReplicaSet creates/destroys pods to match desired count**

🔹 **Step 5: Update Deployment (Rolling Update)**

Change image in YAML:

```
image: booking-app:v2
```

Apply again:

`kubectl apply -f deployment.yaml`

- **Deployment creates new ReplicaSet**
- **Slowly replaces old pods with new pods**
- **Zero downtime**

🔧 Step 6: Delete Deployment

`kubectl delete deployment booking-deployment`

- **Deletes deployment, ReplicaSet, and pods**

🧠 Mental Model

Deployment → ReplicaSet → Pods → Containers

🧠 How to expose a Kubernetes app

You cannot just use containerPort—that only tells Kubernetes what port the container listens on internally.

To access the app from outside the cluster, you use a Service.

🧠 What is a Service in Kubernetes?

A Service is an abstraction that defines a logical set of pods and a policy to access them.

In simpler words:

- **Pods are ephemeral (can die, restart, or move to other nodes)**
- **Services provide a stable network endpoint to reach your pods**

🔥 Why Services?

- **Pods come and go → IPs change**
- **Without Service → you cannot reliably reach your pods**
- **Service = stable access point (DNS + virtual IP)**

🔹 How it works

Browser / Client → Service → Pod(s)

- **Service selects pods via labels**
- **Distributes traffic evenly among pods (like a basic load balancer)**
- **Provides stable IP / DNS name inside cluster**

🔹 Mental Model

[User/Browser] → NodePort → Service → Pod → Container

- Service = stable endpoint / router
- Pod = ephemeral worker

🔥 Key Insights

1. Pods = ephemeral, Services = stable
2. Services use labels to find pods
3. NodePort or LoadBalancer = only for external access
4. ClusterIP = internal-only

1️⃣ Create a Service (NodePort)

Example: expose the booking-app Deployment we created.

```
apiVersion: v1
kind: Service
metadata:
  name: booking-service
spec:
  selector:
    app: booking          # must match Deployment pod labels
  type: NodePort          # exposes port on node
  ports:
    - port: 3000          # internal service port
      targetPort: 3000    # containerPort in pods
      nodePort: 31000     # port you access on your machine
```

⚙️ Explanation

- port → port on the Service inside cluster
- targetPort → pod/container port
- nodePort → port exposed on node for external access
- type: NodePort → simplest way to expose outside cluster

2️⃣ Apply the Service

```
kubectl apply -f service.yaml
```

3️⃣ Access the app

1. Get Minikube node IP:

```
minikube ip
```

2. Access app in browser or curl:

```
http://<minikube-ip>:31000
```

4️⃣ Alternative: kubectl port-forward (dev only)

```
kubectl port-forward deployment/booking-deployment 3000:3000
```

- Local port 3000 → pod containerPort 3000
- Quick way to test locally
- Not for production

5️⃣ Other Service Types

Type	Use case
ClusterIP	Default → internal-only (pods talk to pods)
NodePort	Exposes service on host port
LoadBalancer	Cloud provider LB exposes externally
ExternalName	Maps to external DNS

Kuberneters api

👉 **It’s the brain entry point**

kubectl → Kubernetes API Server → Cluster

Everything you did:

kubectl apply -f deployment.yaml

👉 **actually becomes:**

POST /apis/apps/v1/deployments

⚔️ **Reality**

- **kubectl = wrapper**
- **Kubernetes API = actual system**

1. Everything goes through API Server

kubectl → API Server → etcd

---

2. YAML = API request

Your file:

kind: Deployment

👉 **becomes API object**

3. Cluster stores state in etcd

👉 **API server reads/writes here**

⚔️ **Key Insight**

Kubernetes is API-driven system

Everything = API objects

🧠 **One-line takeaway**

You don’t use Kubernetes
You talk to its API (kubectl just helps)

Step-by-step:

1. kubectl → sends request to API Server
2. API Server → validates + stores in etcd
3. Controller (Deployment Controller) sees desired state
4. Controller creates ReplicaSet
5. ReplicaSet creates Pods
6. Scheduler assigns pods to node
7. Kubelet runs containers

Kubectl

🧠 **What is kubectl?**

👉 **kubectl is a command-line tool (CLI) to talk to a Kubernetes cluster**

⚙️ **What it actually does**

When you run:

kubectl get pods

👉 **It:**

1. Sends HTTP request → Kubernetes API Server
2. API Server responds with data
3. kubectl prints it nicely

You → kubectl → API Server → Cluster

Most imp kubectl commands

📦 **1. Apply / Create Resources**

kubectl apply -f file.yaml

👉 **Creates OR updates resources**
👉 **Most used command**

kubectl delete -f file.yaml

👉 **Deletes everything defined in file**

🔍 **2. Get (View Resources)**

kubectl get pods
kubectl get deployments
kubectl get services

👉 See what’s running

Useful variants:

kubectl get pods -o wide

👉 Shows IP, node

kubectl get all

👉 Everything (pods, svc, deploy)

🧠 3. Describe (Deep Debug)

kubectl describe pod <pod-name>

👉 Shows:

- Events
- Errors
- Why pod failed

📜 4. Logs (VERY IMPORTANT)

kubectl logs <pod-name>

👉 See app logs

Multi-container:

kubectl logs <pod-name> -c <container-name>

💀 5. Delete (Force Actions)

kubectl delete pod <pod-name>

👉 Kubernetes will recreate it

kubectl delete deployment <name>

👉 Removes entire app

📈 6. Scaling

kubectl scale deployment booking-deployment --replicas=5

👉 Increase/decrease pods

🔁 7. Rolling Updates

kubectl set image deployment/booking-deployment booking-app=booking-app:v2

👉 Update app version

kubectl rollout status deployment booking-deployment

👉 Check update progress

kubectl rollout undo deployment booking-deployment

👉 Rollback

🧪 8. Exec (Run inside container)

kubectl exec -it <pod-name> -- /bin/sh

👉 Enter container (like SSH)

🌐 9. Port Forward (Local access)

kubectl port-forward pod/<pod-name> 3000:3000

👉 Access app via localhost

🧩 **10. Services**

`kubectl get svc`

👉 **List services**

`kubectl describe svc <service-name>`

👉 **Debug networking**

⚙️ **11. Config (Cluster switching)**

`kubectl config current-context`

👉 **Which cluster you're using**

`kubectl config use-context <name>`

👉 **Switch cluster**

📊 **12. Watch (Real-time)**

`kubectl get pods -w`

👉 **Live updates**

🔥 **13. Top (Resource usage)**

`kubectl top pods`

👉 **CPU / memory usage**

Minikube

🧠 What is Minikube?

👉 **Minikube is a tool that runs a Kubernetes cluster locally on your machine**

🔥 One-line definition

Minikube = local Kubernetes cluster for development/testing

Most important Minikube command

🟢 1. Start Cluster

```
minikube start --driver=docker
```

👉 **Creates + starts your Kubernetes cluster**

🛑 2. Stop Cluster

```
minikube stop
```

👉 **Stops cluster (frees CPU/RAM, keeps data)**

💀 3. Delete Cluster

minikube delete

👉 **Full cleanup (everything gone)**

🔄 4. Restart (combo)

```
minikube stop
minikube start
```

👉 **Fixes many weird issues**

🌐 5. Access Service (VERY IMPORTANT)

`minikube service <service-name>`

👉 Opens app in browser

🌍 6. Get Cluster IP

`minikube ip`

👉 Used for manual access

📊 8. Dashboard (UI)

`minikube dashboard`

👉 Opens Kubernetes UI in browser

🔍 9. Status

`minikube status`

👉 Shows if cluster is running or stopped

⚙️ 11. Config (resources)

```
minikube config set memory 4096
minikube config set cpus 2
```

👉 Control RAM/CPU usage

🔥 12. Logs (debug cluster)

`minikube logs`

👉 Debug cluster issues (not app logs)

📦 13. Addons (extra features)

minikube addons list
minikube addons enable ingress

👉 **Enable features like Ingress**

🎬 THE STORY: “From Code → Production System” running your first pod

🚀 STEP 1: Your App Exists (The Developer Phase)

You wrote:

```
app.get("/", (req, res) => {
  res.send("Booking Service Running 🚀");
});
```

👉 Right now, this is just:

“Some code sitting on your laptop”

When you run:

```
node index.js
```

👉 You are saying:

“Hey Node.js, run this process on MY machine”

🧠 Problem here

- **Runs only on your laptop ❌**
- **If it crashes → dead ❌**
- **Cannot scale ❌**

🐳 STEP 2: Docker (Packaging Phase)

You create a Dockerfile using Docker

🎯 What Docker actually does

It converts your app into:

📦 A portable box (image)

This box contains:

- Your code
- Node.js runtime
- Dependencies

🐳 STEP 2: Dockerize the App

📄 Create Dockerfile

```
FROM node:18 WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "index.js"]
```

📄 Create .dockerignore

```
node_modules
npm-debug.log
```

When you run:

```
docker build -t booking-app .
```

👉 You are saying:

“Package everything into an image named booking-app”

When you run:

```
docker run -p 3000:3000 booking-app
```

👉 You are saying:

“Start one instance of my app”

🧠 **Still a problem**

- **Only one instance**
- **No auto-restart**
- **No scaling**

☸️ **STEP 3: Enter Kubernetes (The System Phase)**

`minikube start`

You start Minikube A local Kubernetes cluster

🎯 **What just happened**

Minikube created:

🏢 **A mini data center inside your laptop**

Inside it:

- **Control Plane (brain)**
- **Node (worker)**

When you run:

`minikube docker-env | Invoke-Expression`

👉 **You switched from:**

“My laptop Docker”
➡️
“Cluster’s internal Docker”

this command is terminal specific and we can go on another termial it will point to laptop docker

🧠 **Why rebuild image?**

```
docker build -t booking-app .
```

👉 **Now image exists inside cluster, not your laptop**

⚙️ **STEP 4: Deployment (The Manager)**

Create deployment.yaml in root folder

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: booking-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: booking
  template:
    metadata:
      labels:
        app: booking
    spec:
      containers:
        - name: booking-app
          image: booking-app
          imagePullPolicy: Never
          ports:
```

- containerPort: 3000

1. apiVersion: apps/v1

👉 Which Kubernetes API to use

- **apps/v1 → for Deployments (modern standard)**

2. kind: Deployment

👉 What are you creating?

- **Not a pod directly ❌**
- **A Deployment ✅**

🧠 Why Deployment?

Because it gives:

- **Auto-healing**
- **Scaling**
- **Rolling updates**

3. metadata

metadata:
name: booking-deployment

👉 Name of this resource inside cluster

4. spec (MOST IMPORTANT)

This is the desired state

🔁 **4.1 replicas: 2**

👉 **You are saying:**

“I want 2 pods always running”

🔥 **If:**

- **1 pod dies → Kubernetes creates new one**
- **You delete pod → recreated**

🎯 **4.2 selector**

```
selector:
  matchLabels:
    app: booking
```

👉 **Deployment controls pods with this label**

🧩 **4.3 template (VERY IMPORTANT)**

This defines how each pod should look

🔹 **metadata (inside template)**

```
labels:
  app: booking
```

👉 **Every pod will have this label**

⚠️ **Critical rule**

This MUST match selector:

selector.matchLabels = template.metadata.labels

If mismatch → Deployment breaks

🔧 **4.4 spec (inside template)**

👉 **Actual container config**

📦 **Containers**

containers:
- name: booking-app

👉 **Name of container inside pod**

🐳 **image: booking-app**

👉 **Which Docker image to run**

- **Local: booking-app**
- **Production: username/booking-app:v1**

🚫 **imagePullPolicy: Never**

👉 **Kubernetes will:**

“DO NOT pull from internet, use local image”

🧠 **Why you used it**

Because Minikube has access to your local Docker

⚠️ **In production**

imagePullPolicy: Always

🌐 **ports**

containerPort: 3000

👉 **Your app runs on port 3000 inside container**

⚠️ **Important:**

- **This does NOT expose app outside**
- **Only tells Kubernetes internal port**

🧠 **Full Flow (Mental Model)**

You apply this file

kubectl apply → API Server

Kubernetes does:

1. **Deployment created**
2. **Scheduler picks node**
3. **Creates 2 Pods**
4. **Each pod runs:**
    - **booking-app container**
    - **using image booking-app**

You apply:

kubectl apply -f deployment.yaml

👉 **You are NOT creating pods directly**

👉 **You are saying:**

“Hey Kubernetes, I want 2 copies of this app running forever”

🧠 **What Kubernetes does internally**

1. **API Server receives request**
2. **Stores in etcd**
3. **Scheduler picks node**
4. **Kubelet starts containers**

🎯 **Deployment's job**

- **Maintain desired state**
- **If pod dies → recreate**
- **If you scale → adjust count**

This line is CRITICAL:

replicas: 2

👉 **Means:**

"I don't care HOW, just make sure 2 are always running"

Label system

labels:
app: booking

👉 **This is like a tag**

Used later by Service to find pods

🌐 **STEP 5: Service (The Network Layer)**

Service .yaml

👉 **Its job:**

"Expose my pods and give them a stable way to receive traffic"

🔥 **Why Service is needed (don't skip this)**

Without Service:

- **Pods have random IPs ❌**

- **Pods die and get recreated ❌**
- **You can’t reliably hit your app ❌**

👉 Service solves this by acting like a stable entry point

```
apiVersion: v1
kind: Service
metadata:
  name: booking-service
spec:
  type: NodePort
  selector:
    app: booking
  ports:
    - port: 80
      targetPort: 3000
      nodePort: 30007
```

You apply:

```
kubectl apply -f service.yaml
```

1. API Version & Type

apiVersion: v1
kind: Service

👉 You’re telling Kubernetes:

“I want to create a Service resource”

🎯 **What Service does**

Creates a stable entry point

2. Metadata

metadata:
name: booking-service

👉 **Name of your service**

Used for:

- **DNS inside cluster**
- **CLI commands**

3. Spec (the real logic starts here)

🔍 **Type: NodePort**

type: NodePort

👉 **This is CRITICAL**

It means:

“Expose this service outside the cluster using a port on the node”

🧠 **What actually happens**

Kubernetes opens a port on the node:

Node IP:30007

👉 **Any request hitting that goes to your app**

🔍 **Selector (VERY IMPORTANT)**

selector:
app: booking

👉 This connects Service → Pods

🧠 How?

Service says:

“Find all pods with label app=booking”

From your Deployment:

labels:
app: booking

👉 That’s how they get linked

🔍 Ports section (core networking)

ports:
- port: 80
targetPort: 3000
nodePort: 30007

🔹 port: 80

👉 Service’s internal port

Inside cluster:

Service → port 80

🔹 targetPort: 3000

👉 Where your app is actually running

Your Express app:

const PORT = 3000;

So:

Service → forwards → Pod:3000

🔹 **nodePort: 30007**

👉 **External access point**

You access app via:

NodeIP:30007

🔥 **Full Request Flow (THIS is the real understanding)**

User (browser)
↓
Node IP:30007
↓
Service (port 80)
↓
Pod (port 3000)
↓
Express app

🧠 **Key concept: Load Balancing**

If you have:

replicas: 2

👉 **Service will:**

- **Distribute traffic between both pods**
- **Round-robin style**

⚠️ **Important things most people miss**

---

1. nodePort range

- **Must be between 30000–32767**
- **That's why you used 30007**

2. You usually don't set nodePort manually

👉 **Kubernetes can auto-assign it**

3. NodePort is NOT used in production

👉 **It's mainly for:**

- **Local dev (like Minikube)**
- **Testing**

In real world:

- **Use LoadBalancer / Ingress**

When you run:

minikube service booking-service

👉 **It opens browser + sets routing**